IP- Sp 26 Finals Model Solution

Q No	KEY	Q No	KEY	Q No	KEY	Q No	KEY	Q No	KEY
1	B	11	A	21	B	31	B	41	B
2	C	12	B	22	B	32	B	42	B
3	B	13	A	23	B	33	B	43	D
4	A	14	B	24	B	34	B	44	C
5	B	15	A	25	B	35	B	45	B
6	B	16	A	26	B	36	B	46	C
7	B	17	B	27	C	37	B	47	B
8	B	18	B	28	C	38	B	48	D
9	B	19	A	29	A	39	A	49	B
10	C	20	D	30	B	40	A	50	B

Q2

Solution Part A: RGB to CMY to CMYK Conversion

Given: R = 0.8, G = 0.4, B = 0.2

a) RGB to CMY:

- C = 1 - R = 1 - 0.8 = **0.2** 1
- M = 1 - G = 1 - 0.4 = **0.6** 1
- Y = 1 - B = 1 - 0.2 = **0.8** 1

Total = 6.

b) CMY to CMYK:

- K = min(C, M, Y) = min(0.2, 0.6, 0.8) = **0.2** 1
- Since K = 0.2 < 1:
  - C' = (C - K)/(1 - K) = (0.2 - 0.2)/(1 - 0.2) = 0/0.8 = **0**
  - M' = (M - K)/(1 - K) = (0.6 - 0.2)/(1 - 0.2) = 0.4/0.8 = **0.5**
  - Y' = (Y - K)/(1 - K) = (0.8 - 0.2)/(1 - 0.2) = 0.6/0.8 = **0.75**

2

Final CMYK: C'=0, M'=0.5, Y'=0.75, K=0.2

Part B: Color Space Properties

a) Additive vs Subtractive:

- **RGB (Additive):** Colors are created by adding light. Starting from black (no light), adding R, G, B creates colors. R+G+B = White. Used in displays/monitors that emit light. 1
- **CMY (Subtractive):** Colors are created by subtracting wavelengths from white light. Starting from white (paper/canvas), adding C, M, Y absorbs light. C+M+Y = Black. Used in printing where inks absorb light. 1

Total = 4

b) H=0°, S=1.0, I=0.5:

- This represents **Pure Red** (Hue=0° is red, S=1.0 means fully saturated, I=0.5 means medium brightness) 1

c) Increasing I from 0.5 to 0.8:

- The color becomes **brighter/lighter** (more luminous)
- It's equivalent to adding white light to the pure red 1

- The color appears as a lighter shade of red (pink-ish)
- Hue remains red, saturation remains full, only brightness increases

Q3

Q3. You have a grayscale thermal image where intensity represents temperature:

- 0-63: Cold (should map to Blue)
- 64-127: Moderate (should map to Green)
- 128-191: Warm (should map to Yellow)
- 192-255: Hot (should map to Red)

a) Design a simple mapping function for the R channel. Express as piecewise function.
b) Why is pseudo-coloring useful for grayscale images? Give TWO specific reasons.
c) Given transformation matrix from RGB to YCbCr:

```
[Y ]   [ 0.299   0.587   0.114] [R]
[Cb] = [-0.169  -0.331   0.500] [G]
[Cr]   [ 0.500  -0.419  -0.081] [B]
```

For RGB = [255, 128, 64], Calculate Y, Cb, Cr values.

Q3

Solution

a) R Channel Piecewise Function:

```
R(I) = {
    0,                    for 0 ≤ I ≤ 63      (Blue - no red)
    0,                    for 64 ≤ I ≤ 127    (Green - no red)
    (I - 128) / 63,       for 128 ≤ I ≤ 191   (Yellow - ramp up red)
    255,                  for 192 ≤ I ≤ 255   (Red - full red)
}
```

3

Or using linear scaling:

```
R(I) = {
    0,                    for 0 ≤ I ≤ 63
    0,                    for 64 ≤ I ≤ 127
    4×(I - 128),          for 128 ≤ I ≤ 191   (scales 0→252)
    255,                  for 192 ≤ I ≤ 255
}
```

b) Two Reasons for Pseudo-Coloring:

1. **Enhanced Visual Discrimination:** Human eye can distinguish ~30 shades of gray but millions of colors. Pseudo-coloring allows detection of subtle intensity variations that would be invisible in grayscale (e.g., detecting small temperature differences in thermal images).
2. **Intuitive Interpretation:** Colors can convey meaning naturally (blue=cold, red=hot). This makes analysis faster and more intuitive, especially in medical imaging (highlighting tumors), geothermal studies, or scientific visualization.

3

c) RGB to YCbCr Conversion:

Given: RGB = [255, 128, 64]

Y Calculation: Y = 0.299×255 + 0.587×128 + 0.114×64 Y = 76.245 + 75.136 + 7.296 **Y = 158.677 ≈ 158.68**

Cb Calculation: Cb = -0.169×255 + (-0.331)×128 + 0.500×64 Cb = -43.095 - 42.368 + 32.0 **Cb = -53.463 ≈ -53.46**

Cr Calculation: Cr = 0.500×255 + (-0.419)×128 + (-0.081)×64 Cr = 127.5 - 53.632 - 5.184 **Cr = 68.684 ≈ 68.68**

Final: Y = 158.68, Cb = -53.46, Cr = 68.68

4

Q4 a) A binary image contains straight lines oriented horizontally, vertically, at 45°, and at −45°.

Given a set of 3 x3 kernels that can be used to detect one-pixel breaks in these lines. Assume that the intensities of the lines and background are 1 and 0, respectively. How would you modify the Sobel and Prewitt kernels so that they give their strongest gradient response for edges oriented at ± 45.

b) Intensity values of a section of horizontal scan line

6	7	6	5	4	3	3	3	3	2	2	2	2	2	8	7	6	9	5

All of the following questions are based on the above section of horizontal scan line.

a. Draw a line plot of its intensity values and show all data points and label ramp and step function.
b. What is the maximum value in the 1st derivative output of the above scan line?
c. What is the minimum value in the 1st derivative output of the above scan line?
d. What is the minimum value in the 2nd derivative output of the above scan line?
e. How many 0's in the 2nd derivative output of the above scan line?
f. How many 0's in the 1st derivative output of the above scan line?

Q4

Solution

Part a: Modified Sobel/Prewitt for ±45° Edges [4 marks]

Original Sobel (detects vertical edges):

```
[-1  0  1]    [-1 -2 -1]
[-2  0  2]    [ 0  0  0]
[-1  0  1]    [ 1  2  1]
```

→ 1

Modified for +45° diagonal edges:

```
[ 0  1  2]    [-2 -1  0]
[-1  0  1]    [-1  0  1]
[-2 -1  0]    [ 0  1  2]
```

→ 1

Modified for -45° diagonal edges:

```
[-2 -1  0]    [ 0  1  2]
[-1  0  1]    [-1  0  1]
[ 0  1  2]    [-2 -1  0]
```

→ 1

For line break detection (3×3 kernels):

```
Horizontal:      Vertical:      +45°:          -45°:
[1 1 1]          [1 0 1]        [1 0 0]        [0 0 1]
[0 0 0]          [1 0 1]        [0 0 0]        [0 0 0]
[1 1 1]          [1 0 1]        [0 0 1]        [1 0 0]
```

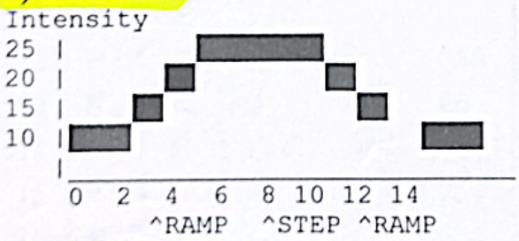
→ 1

Center pixel = 0 indicates break when surrounded by 1's.

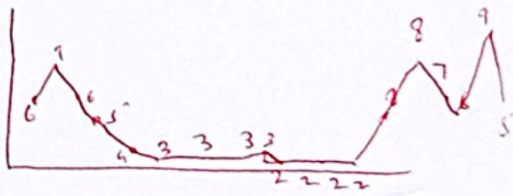
Part b: Scan Line Analysis

Given scan line: **10, 10, 10, 10, 15, 20, 25, 25, 25, 25, 20, 15, 10, 10, 10**

a) Line Plot:



1



- **Ramp:** positions 4-7 (increasing)
- **Step:** positions 7-10 (constant at 25)
- **Ramp:** positions 10-13 (decreasing)

b) Maximum value in 1st derivative: Answer: 5 (occurs at positions 4-6 where intensity increases by 5 per step)

6

1

c) Minimum value in 1st derivative: **Answer: -5** (occurs at positions 10-12 where intensity decreases by 5 per step)

d) Minimum value in 2nd derivative: Answer: -5 (at position 10 where derivative changes from 0 to -5)

e) Number of 0's in 2nd derivative: Answer: 11 (all constant regions: positions 0-3, 7-9, 13-14)

f) Number of 0's in 1st derivative: Answer: 7 (constant regions: positions 0-3 and 7-10 and 13-14)

Q5. Encode the string: **"TOBEORNOTTOBEORNOT" using LZW**

Initial dictionary (assume ASCII):

- Index 65-90: A-Z
- Index 256 onwards: new sequences

a) Show the complete encoding process step by step. For each step, indicate:
Use a table format:

Step	Input	W	W+C In Dict?	Output	New Entry	Next Index
1	T	T	-	84	-	256
...	...	...				

b) Write the final compressed output as a sequence of codes.

c) Calculate the compression ratio if each character is originally 8 bits and each code is 12 bits.

Solution

a) Complete Encoding Process:

Step	Input	W	W+C in Dict?	Output	New Entry	Next Index
1	T	""	-	-	-	256
2	O	T	No	84	TO(256)	257
3	B	O	No	79	OB(257)	258
4	E	B	No	66	BE(258)	259
5	O	E	No	69	EO(259)	260
6	R	O	No	79	OR(260)	261
7	N	R	No	82	RN(261)	262
8	O	N	No	78	NO(262)	263
9	T	O	No	79	OT(263)	264
10	T	T	Yes	-	-	264
11	O	TO	Yes	-	-	264
12	B	TOB	No	256	TOB(264)	265
13	E	B	No	66	BE(already)	265
14	O	E	Yes	-	-	265
15	R	EO	No	259	EOR(265)	266
16	N	R	Yes	-	-	266
17	O	RN	No	261	RNO(266)	267
18	T	O	Yes	-	-	267
19	EOF	OT	Yes	263	-	-

b) Final Compressed Output:

[84, 79, 66, 69, 79, 82, 78, 79, 256, 66, 259, 261, 263]

c) Compression Ratio:

- **Original:** 18 characters × 8 bits = 144 bits
- **Compressed:** 13 codes × 12 bits = 156 bits
- **Compression Ratio:** 144/156 = **0.923:1** (actually expansion, not compression in this case!)
- Note: LZW works better with longer strings with more repetition

Q6. Arithmetic Coding - Encoding

Given a source with three symbols and their probabilities:

- X: P(X) = 0.5, interval [0, 0.5)
- Y: P(Y) = 0.3, interval [0.5, 0.8)
- Z: P(Z) = 0.2, interval [0.8, 1.0)

Encode the message: **"XYZX" using** Arithmetic encoding

a) Show the interval narrowing for each symbol step by step:

Initial interval: [0, 1)

b) Choose any number within the final interval to represent the message.

c) Convert your chosen number to binary (at least 4 bits after decimal point).

d) How many bits does this arithmetic code use compared to 2-bit fixed-length coding for 3 symbols?

Solution

a) Interval Narrowing:

Initial: [0, 1)

After X (P=0.5, range [0, 0.5)):

- New range = 0 + 0.5×(1-0) = 0.5
- Interval: **[0, 0.5)**

After Y (P=0.3, range [0.5, 0.8)):

- Start = 0 + 0.5×(0.5-0) = 0.25
- End = 0 + 0.8×(0.5-0) = 0.4
- Interval: **[0.25, 0.4)**

After Z (P=0.2, range [0.8, 1.0)):

- Start = 0.25 + 0.8×(0.4-0.25) = 0.25 + 0.12 = 0.37
- End = 0.25 + 1.0×(0.4-0.25) = 0.25 + 0.15 = 0.4
- Interval: **[0.37, 0.4)**

After X (P=0.5, range [0, 0.5)):

- Start = 0.37 + 0×(0.4-0.37) = 0.37
- End = 0.37 + 0.5×(0.4-0.37) = 0.37 + 0.015 = 0.385
- **Final Interval: [0.37, 0.385)**

b) Chosen Number:

Any number in [0.37, 0.385), for example: **0.38**

c) Binary Conversion of 0.38:

- 0.38 × 2 = 0.76 → 0
- 0.76 × 2 = 1.52 → 1
- 0.52 × 2 = 1.04 → 1
- 0.04 × 2 = 0.08 → 0
- 0.08 × 2 = 0.16 → 0
- 0.16 × 2 = 0.32 → 0

Binary: 0.011000... ≈ 0.0110

d) Bit Comparison:

- **Arithmetic code:** ~6 bits (0.011000 requires about 6 bits for sufficient precision)
- **Fixed-length:** 4 symbols × 2 bits = 8 bits (since 3 symbols need $\log_2 3 \approx 2$ bits, rounded up)
- **Savings:** 8 - 6 = 2 bits saved (25% compression)

Q7. A) Consider the following CNN architecture:
Layer 1: Input image 224×224×3
Layer 2: CONV with 64 filters, kernel size 7×7, stride 2, padding 3
Layer 3: Max pooling 3×3, stride 2
Layer 4: CONV with 128 filters, kernel size 3×3, stride 1, padding 1
Layer 5: Max pooling 2×2, stride 2
a) Calculate the output spatial dimensions after each layer.
b) Calculate the number of parameters (weights + biases) in Layer 2 and Layer 4.
c) If we use ReLU activation after each CONV layer, does it add any trainable parameters? Explain.

Q7

Solution

Part A: CNN Dimension Calculations

Formula: Output = (Input - Kernel + 2×Padding)/Stride + 1

a) Output Dimensions:

Layer 1 (Input): 224×224×3

Layer 2 (CONV 64, 7×7, stride 2, pad 3):

- (224 - 7 + 2×3)/2 + 1 = (224 - 7 + 6)/2 + 1 = 223/2 + 1 = 111.5 + 1 = 112
- **Output: 112×112×64**

Layer 3 (MaxPool 3×3, stride 2):

- (112 - 3)/2 + 1 = 109/2 + 1 = 54.5 + 1 = 55
- **Output: 55×55×64**

Layer 4 (CONV 128, 3×3, stride 1, pad 1):

- (55 - 3 + 2×1)/1 + 1 = (55 - 3 + 2)/1 + 1 = 54 + 1 = 55
- **Output: 55×55×128**

Layer 5 (MaxPool 2×2, stride 2):

- (55 - 2)/2 + 1 = 53/2 + 1 = 26.5 + 1 = 27
- **Output: 27×27×128**

b) Parameter Calculations:

Layer 2: 64 filters of 7×7×3

- Parameters = (7×7×3 + 1) × 64 = (147 + 1) × 64 = **9,472 parameters**

Layer 4: 128 filters of 3×3×64

- Parameters = (3×3×64 + 1) × 128 = (576 + 1) × 128 = **73,856 parameters**

c) ReLU Parameters:

- **No trainable parameters**
- ReLU is element-wise: f(x) = max(0, x)
- No weights or biases to learn
- Only adds computational operations, not parameters

Q8 A). You are designing a CNN for medical image analysis where:

- Input: Grayscale X-ray images (normalized to [0, 1])

- Task: Binary classification (disease present or not)
- Network: 5 convolutional layers + 2 fully connected layers

a) Which activation function would you use in the hidden layers? Justify your choice.
b) Which activation function would you use in the output layer? Why?
c) Would you consider using ELU instead of ReLU? Discuss one advantage and one disadvantage.

B) You need to choose an object detection algorithm for the following scenarios:
Scenario A: Real-time pedestrian detection on an autonomous vehicle (30 FPS required)
Scenario B: Medical image analysis where precision is critical (speed not important)
Scenario C: Detecting small objects (e.g., birds) in aerial images
For each scenario:

- Choose between: Faster R-CNN, YOLO, or RetinaNet
- Justify your choice with specific architectural advantages

Solution

Part A: Medical X-ray CNN Design

a) Hidden Layer Activation Function:

- **Choice: ReLU (Rectified Linear Unit)**
- **Justifications:**
  1. **Avoids vanishing gradient:** Important for 5-layer network depth
  2. **Computationally efficient:** f(x) = max(0,x) is simple
  3. **Sparse activation:** Many neurons = 0, leading to efficient representations
  4. **Works well with grayscale:** No issues with normalized [0,1] input
  5. **Proven in medical imaging:** Standard choice for X-ray analysis CNNs

b) Output Layer Activation Function:

- **Choice: Sigmoid**
- **Why:**
  1. **Binary classification:** Outputs probability in [0, 1]
  2. **Interpretable:** Direct probability of disease presence
  3. **Loss function compatibility:** Works with binary cross-entropy loss
  4. **Single neuron output:** Perfect for binary (disease/no disease)

c) ELU vs ReLU Consideration:

ELU Advantage:

- **Negative values:** ELU(x) = x for x>0, α(e^x - 1) for x≤0
- Helps with mean activation closer to zero, potentially faster convergence
- Reduces bias shift, may improve feature learning

ELU Disadvantage:

- **Computational cost:** Exponential function is slower than max(0,x)
- In medical imaging with limited data, the marginal improvement may not justify the computational overhead
- ReLU's simplicity is often sufficient for X-ray analysis

Part B: Object Detection Algorithm Selection

Scenario A: Real-time Pedestrian Detection (30 FPS)

- **Choice: YOLO (You Only Look Once)**
- **Justification:**
  - Single-stage detector: processes entire image in one forward pass
  - Speed: Can achieve 45-60 FPS on modern GPUs
  - Real-time performance: Designed specifically for speed
  - Good enough accuracy for pedestrian detection

- Global reasoning: Sees entire image context, reducing false positives

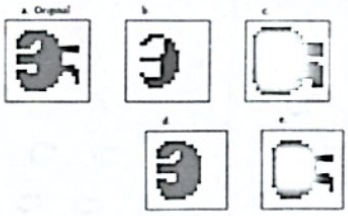
Scenario B: Medical Image Analysis (Precision Critical)

- **Choice: Faster R-CNN**
- **Justification:**
  - Two-stage detector: RPN generates proposals, then refined classification
  - Higher accuracy: Better precision/recall for critical medical diagnosis
  - Handles small lesions/abnormalities well
  - Region-based: Can focus on suspicious areas
  - Speed not important: Can take seconds per image if accuracy is maximized
  - Proven in medical imaging research

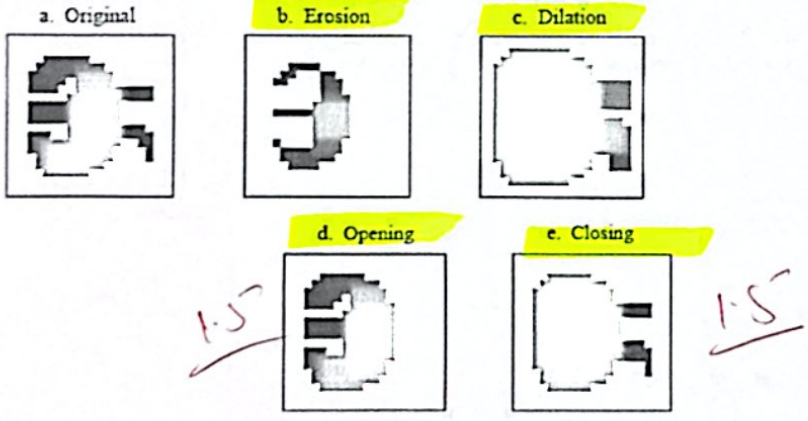
Scenario C: Small Objects in Aerial Images (Birds)

- **Choice: RetinaNet**
- **Justification:**
  - Feature Pyramid Network (FPN): Multi-scale feature representation
  - Excellent for small objects: FPN captures fine details at multiple scales
  - Focal Loss: Addresses class imbalance (many background vs few birds)
  - One-stage but accurate: Better than YOLO for small objects
  - Balances speed and accuracy: Faster than Faster R-CNN, more accurate than YOLO for small targets
  - Specifically designed to handle the "small object problem"
-

Q9) Given original image and four output images each with a different image morphology operation. Label the output images each with correct image morphology operations on answer sheet.



Solution



Q10: Answer the following: [10+10=20]

Part A: Given an image as below, and seed value =22, apply region-based segmentation on it based on region growing approach.

22	24	27	30	33	36	39
23	25	28	31	35	38	41
24	26	29	34	39	43	46
25	27	31	40	48	52	55
26	29	35	47	58	61	64
28	32	40	53	63	67	70
30	35	44	57	68	72	75

Part B: use same image in Part A and apply region-based segmentation using region splitting approach.

10

Solution:

Region Growing.

$|Seed - pixel\ value| \leq 10$

fixed seed

updating Seeds

final

1	1	1	1	0	0	0
1	1	1	1	0	0	0
1	1	1	0	0	0	0
1	1	1	0	0	0	0
1	1	0	0	0	0	0
1	1	0	0	0	0	0
1	0	0	0	0	0	0

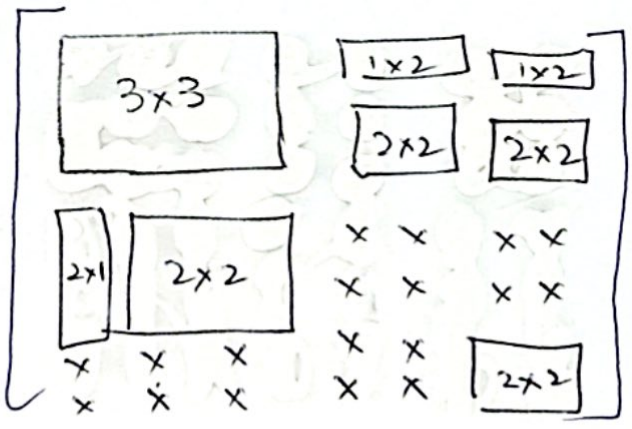
10

all 1s

Region Splitting

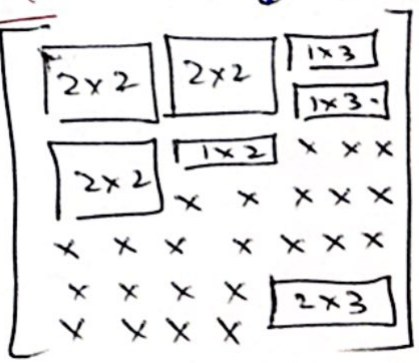
$\max(Region) - \min(Region) \leq 10$

final



10

each alphabet is a region



depends on Split